


# Estructuras de Datos PECL

---

**SIMULACIÓN DE TRIAJE Y ATENCIÓN A PACIENTES EN LOS QUIRÓFANOS DE UN HOSPITA**

03 / 10 / 2026

**Integrantes - Grupo Nº 20:**
Nathan Jurado Díaz - **49227414P**
Leonardo Daniel Pérez Camacho - **Y8955006Q**

**Profesores del curso:**
Carlos Javier Hellín Asensio
Abdelhamid Tayebi Tayebi

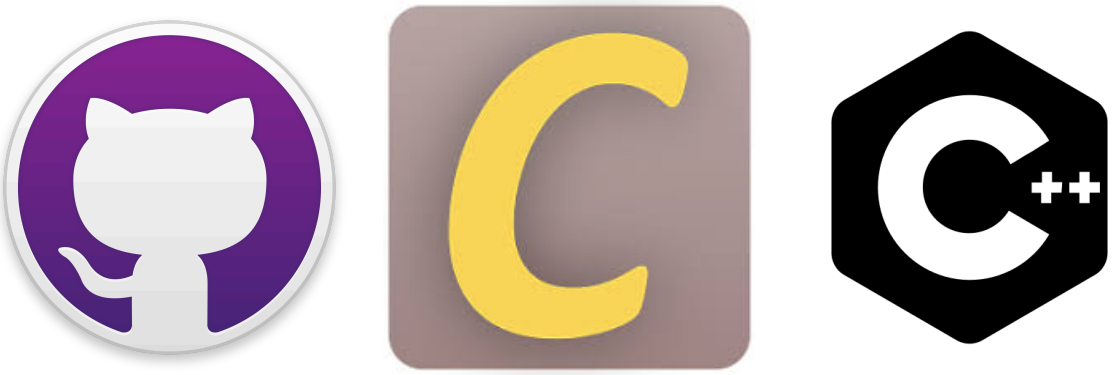
Universidad de Alcalá

# Índice

**Índice** ..... **1**
**Detalles y justificación de la implementación** ..... **2**
**Especificación concreta de la interfaz de los TAD's implementados** ..... **3**
TAD`s creados ..... 3
Definición de las operaciones del TAD (Nombre, argumentos y retorno) ..... 4
**Explicación del comportamiento del programa** ..... **9**
**Explicación de los métodos más destacados** ..... **12**
**Bibliografía** ..... **15**

# Detalles y justificación de la implementación

El entorno en el que se desarrolló el proyecto fue **CodeLite** siendo el lenguaje **C++** teniendo en cuenta el enfoque orientado a objetos, además con el fin de facilitar el desarrollo de la aplicación se hizo uso de la herramienta **GitHub Desktop** para un entorno de trabajo más ordenado y controlado evitando tener problemas de conflicto de código **(merge conflict)** teniendo también así un historial con las modificaciones que se realizaron.



El código se encuentra dividido en distintas **clases**, cada una con sus propios **métodos** y **atributos**, esto con el fin de facilitar la organización y estructura del código, además de la reutilización de funciones y depuración de errores/bugs de manera aislada.

- **main**
- **Controlador**
- **Paciente**
- **Cola**
- **Lista**

- **NodoCola**
- **NodoLista**
- **NodoPila**
- **Pila**

La clase **main** únicamente ejecuta funciones de la clase **Controlador** actuando así controlador como el núcleo principal en el cual se ejecutarán las funciones de otras clases como lo son: **Paciente**, **Cola**, **Lista** entre otros.

Mientras que las clases **Pila**, **Cola** y **Lista** las utilizaremos para el almacenamiento de los pacientes, dependiendo de en qué fase se encuentre el paciente estas tendrán funciones para el tratamiento de los pacientes.

**Pila** se encarga de almacenar temporalmente pacientes generados de forma aleatoria hasta un máximo de 48, **Cola (Salas)** gestiona la llegada de los pacientes con un método **FIFO** mientras esperan su turno para entrar al quirófano y **Lista (Quirófanos)** permite una más avanzada de los paciente.

Universidad de Alcalá

# Especificación concreta de la interfaz de los TAD's implementados

## TAD`s creados

| Paciente | Pila | Cola | Lista | Controlador |
|---|---|---|---|---|

- **Paciente:** Modela los atributos de un paciente (DNI, identificador/prioridad, tipo de enfermedad y habitación asignada) el identificador marca también la prioridad del paciente, así dejamos todos los datos y funciones lógicas del paciente, incluyendo la generación de datos aleatorios como lo son su DNI en esta clase.

- **Pila:** Modela una estructura lineal **FIFO** (First In, First Out) utilizada para gestionar las salas de preoperatorio se utiliza en el controlador como el depósito inicial donde se almacenan temporalmente los paciente como **máximo 48 pacientes** generados de forma aleatoria, antes de ser derivados a las salas de preoperatorio, es decir las **Colas**.

- **Colas:** También maneja una estructura **FIFO** representando las salas de preoperatorio del hospital es decir las **Salas (A, B ,C y D)** permitiendo gestionar la llegada de los pacientes de la **Pila** y mantener un orden y equilibrio en cuando el número de pacientes en una sala.

- **Lista:** Modela una lista enlazada dinámica utilizada para la gestión de los quirófanos, teniendo la ventaja de que con esta podemos tener punteros hacia la cabecera o el final de la lista, la empleamos para la llegada de los pacientes al quirófano dando la ventaja de que podemos gestionar el **orden** de llegada de dichos pacientes dependiendo de la gravedad, esto también nos deja que podamos hacer búsquedas dentro de ellas, por ejemplo: buscar al paciente con mayor **prioridad/gravedad** o el primero y último de la **Lista**

- **Controlador:** Modela la lógica de control principal del flujo del programa y el menú de opciones. Se muestra la interfaz de usuario y gestiona el flujo de opciones elegidas por el usuario.

Universidad de Alcalá

# Definición de las operaciones del TAD (Nombre, argumentos y retorno)

**Paciente**: Ignorando los *Getters* y *Setters* que tiene la clase Paciente…

**saludarPacientePila()**

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Muestra por consola los datos básicos del paciente (DNI y enfermedad) al ser consultado desde la pila.

**saludarPacienteCola()**

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Muestra por consola los datos detallados del paciente (DNI, enfermedad, ID y habitación)

**randomDni()**

**Argumentos**: Ninguno

**Retorno**: string

**Descripción**: Genera de forma aleatoria un DNI de 8 dígitos y una letra.

**randomEnfermedad()**

**Argumentos**: Ninguno

**Retorno**: bool

**Descripción**: Genera aleatoriamente el tipo de enfermedad (**True** o **False**)

**pacienteVacio()**

**Argumentos**: Ninguno

**Retorno**: Paciente

**Descripción**: Devuelve un objeto paciente configurado como centinela (con **ID** -1) para representar la ausencia de datos o estructuras vacías.

**Pila**:

### insertar()

**Argumentos**: Paciente p

**Retorno**: void

**Descripción**: Inserta un nuevo paciente en la cima de la pila (apilar), actualizando el puntero al último nodo y aumentando la longitud.

### extraer()

**Argumentos**: Ninguno

**Retorno**: Paciente

**Descripción**: Extrae y elimina el paciente situado en la cima de la pila (**desapilar**), liberando la memoria de su nodo. Si la pila está vacía, devuelve un paciente centinela con ID -1.

### cima()

**Argumentos**: Ninguno

**Retorno**: Paciente

**Descripción**: Consulta y devuelve el paciente que se encuentra en la cima de la pila sin extraerlo. Si está vacía, devuelve el paciente centinela.

### mostrar()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Recorre la pila desde la cima hasta el fondo mostrando los datos básicos de todos los pacientes mediante el método de saludo de la pila.

### getLongitud()

**Argumentos**: Ninguno

**Retorno**: int

**Descripción**: Devuelve el número total de pacientes almacenados actualmente en la pila.

### getUltimo()

**Argumentos**: Ninguno

**Retorno**: pnodoPila (puntero a NodoPila)

**Descripción**: Devuelve el puntero al nodo situado en la cima (último) de la pila.

### getPacienteId()

**Argumentos**: int id

**Retorno**: Paciente

**Descripción**: Recorre la pila desde la cima buscando y devolviendo al paciente cuyo ID coincida con el indicado. Si no lo encuentra, devuelve el paciente centinela.

### getPacienteHabitacion()

**Argumentos**: int habitación

**Retorno**: Paciente

**Descripción**: Recorre la pila buscando y devolviendo al paciente que tenga asignado el número de habitación especificado. Si no existe, devuelve el paciente centinela.

**Cola**:

### insertar()

**Argumentos**: Paciente p

**Retorno**: void

**Descripción**: Inserta un paciente al final de la cola.

### eliminar()

**Argumentos**: Ninguno

**Retorno**: Paciente

**Descripción**: Extrae y devuelve el primer paciente de la cola, liberando la memoria de su nodo (desencolar). Si está vacía, devuelve un paciente centinela.

### eliminar()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Muestra por pantalla la lista completa de pacientes en la cola desde el primero hasta el último.

### verPrimero()

**Argumentos**: Ninguno

**Retorno**: Paciente

**Descripción**: Consulta y devuelve el paciente del frente de la cola sin extraerlo.

### getLongitud()

**Argumentos**: Ninguno

**Retorno**: int

**Descripción**: Devuelve la cantidad de pacientes almacenados en la cola.

### getPrimero()

**Argumentos**: Ninguno

**Retorno**: pnodoCola (puntero a NodoCola)

**Descripción**: Devuelve el puntero directo al primer nodo de la cola.

### getPaciente()

**Argumentos**: int habitación

**Retorno**: Paciente

**Descripción**: Busca y devuelve el paciente que coincida con el número de habitación dado.

### getPacienteId()

**Argumentos**: int id

**Retorno**: Paciente

**Descripción**: Busca y devuelve el paciente con el **ID** solicitado.

### borrarPorId()

**Argumentos**: int id

**Retorno**: void

**Descripción**: Busca, desengancha de la cola y elimina de memoria el nodo del paciente con ese **ID** específico.

**Lista**:

### insertarHead()

**Argumentos**: Paciente p

**Retorno**: void

**Descripción**: Inserta un paciente al inicio (cabecera) de la lista.

### insertarUltimo()

**Argumentos**: Paciente p

**Retorno**: void

**Descripción**: Inserta un paciente al final de la lista.

### eliminarHead()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Elimina y libera de la memoria el primer nodo de la lista.

### eliminarUlitmo()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Elimina y libera de la memoria el último nodo de la lista.

### mostrar()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Muestra por consola todos los pacientes presentes en la lista.

### verHead()

**Argumentos**: Ninguno

**Retorno**: Paciente

**Descripción**: Devuelve el paciente situado en la cabecera sin borrarlo.

### verUltimo()

**Argumentos**: Ninguno

**Retorno**: Paciente

**Descripción**: Devuelve el paciente situado al final de la lista sin borrarlo.

### borrar()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Vacía totalmente la lista, liberando todos sus nodos.

### getLongitud()

**Argumentos**: Ninguno

**Retorno**: int

**Descripción**: Devuelve el número total de nodos/pacientes que componen la lista.

### encontrar()

**Argumentos**: bool enfermedad, bool buscarMayor

**Retorno**: Paciente

**Descripción**: Recorre la lista filtrando por tipo de enfermedad y devuelve el paciente con mayor o menor **ID** (según el booleano buscarMayor).

### getPacienteHabitacion()

**Argumentos**: int habitación

**Retorno**: Paciente

**Descripción**: Recorre la lista y devuelve el paciente que ocupe la habitación indicada.

**Controlador**:

### status()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Imprime el estado global del hospital (cuántos pacientes hay en cada pila, cola y lista) y despliega el menú de opciones principales.

### opcionA()

**Argumentos**: Ninguno

**Retorno**: void

**Descripción**: Métodos que ejecutan las operaciones del sistema (generar pacientes, mover entre colas/listas, limpiar salas, etc.).

### elegirOpcion()

**Argumentos**: string opcion

**Retorno**: void

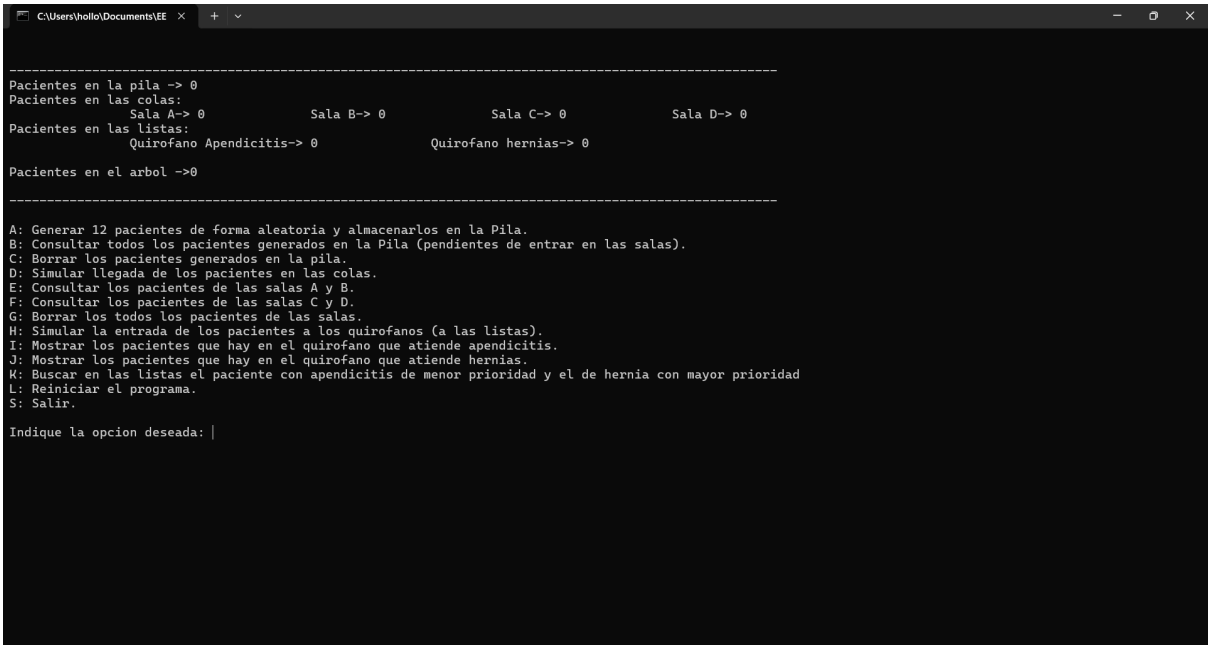
**Descripción**: Evalúa la opción elegida por el usuario y ejecuta el método correspondiente de la opción-A a la opción-L.

# Explicación del comportamiento del programa

El programa consiste en simular la estructura de datos de un paciente que entra a una sala de espera para luego entrar al quirófano y ser atendido por una enfermedad, en este caso las 2 enfermedad son: **Apendicitis** y **Hernias**, dichas salas de espera consisten en 2 colas distintas para cada enfermedad.

- Sala **A** y **B** para los pacientes con **Apendicitis**.
- Sala **C** y **D** para los pacientes con **Hernias**.

El programa tiene además una de opciones las cuales los usuarios pueden elegir ya sea para Generar, Consultar o incluso Borrar pacientes, además de las opción para simular la llegada de dichos pacientes a las colas **(Salas)** al ejecutarse el programa esta será la vista principal que tendremos:




Una consola con datos y opciones, tenemos en la cabeza el número de pacientes en **Pila**, es decir los pacientes que tenemos almacenados para la simulación, luego otra lista con el número de pacientes en las salas, las 2 **Colas** que dependiendo de la enfermedad del paciente se le asignará una sala u otra.

Por último los quirófanos respectivos para cada enfermedad, una **Lista** con los pacientes que vas a ser atendidos para sus respectivas enfermedades.

**El último apartado que vemos en la cabeza de la consola pertenece a la Práctica II la cual se desarrollará más adelante…**

*Pacientes en el arbol -> 0*

Pasando ahora con las **opciones** tenemos un total de **13** para esta primera parte, repartidas como se muestra en la imagen desde la **A** hasta la **S**, vamos una a una explicando que hacen:

- **A: Generar 12 pacientes de forma aleatoria y almacenarlos en la Pila.**

Genera de manera aleatoria 12 pacientes, cada paciente es generado con un DNI, una enfermedad, un número de habitación y por último un identificador, los 2 primeros atributos son generados por la propia clase paciente y los últimos 2 por la clase controlador una vez generados los almacena en **Pila**.

- **B: Consultar todos los pacientes generados en la Pila (pendientes de entrar en las salas).**

Muestra todos los pacientes almacenados en **Pila** usando una función que se encuentra en **pacientes** para mostrar todos los pacientes almacenados en la **Pila** hasta llegar al último almacenado.

- **C: Borrar los pacientes generados en la pila.**

Saltamos nuevamente a la **Pila** y con la función extraer de la misma clase **Pila** vamos sacando uno a uno los pacientes almacenados en la **Pila** hasta que esta esté completamente vacía.

- **D: Simular llegada de los pacientes en las colas.**

Extraemos los pacientes de la **Pila** y dependiendo de la enfermedad que tenga se le asigna a una de las 2 salas de cada enfermedad, es decir que extraemos el paciente de la **Pila** y lo insertamos en la **Cola**, la distribución de los pacientes en las colas está hecha de tal manera las salas queden equilibradas, es decir, si tenemos 10 pacientes con apendicitis se reparten la mitad entre las 2 salas correspondientes a dicha enfermedad.

- **E: Consultar los pacientes de las salas A y B.**

Muestra los pacientes almacenados en la **Cola** con la enfermedad de apendicitis, haciendo uso de la función mostrar de la clase **Cola**.

- **F: Consultar los pacientes de las salas C y D.**

Muestra los pacientes almacenados en la **Cola** con la enfermedad de hernias, haciendo uso de la función mostrar de la clase **Cola**.

- **G: Borrar los todos los pacientes de las salas.**

Saltamos a las **Colas** y se va eliminando los pacientes hasta que las salas están completamente vacías, haciendo uso del método eliminar de la clase **Cola**.

- **H: Simular la entrada de los pacientes a los quirofanos (a las listas).**

Recorremos las salas y dependiendo del identificador que tenga el paciente se le meterá en la **Lista** de los quirófanos en orden dependiendo de su id, siendo el id de mayor gravedad de este siendo además el último agregado en la **Lista**, hacemos esto para cada una de las **Colas**, es decir pasamos de manera ordenada todos los pacientes de las Salas A y B a la Lista del quirófano de su respectiva enfermedad.

Una vez insertados en las Listas los pacientes se va limpiando de las salas los pacientes por su ID dejando las **Colas** libre de pacientes.

- **I: Mostrar los pacientes que hay en el quirofano que atiende apendicitis.**

Muestra los pacientes almacenados en la **Lista** con la enfermedad de apendicitis, haciendo uso de la función mostrar de la clase **Lista**.

- **J: Mostrar los pacientes que hay en el quirofano que atiende hernias.**

Muestra los pacientes almacenados en la **Lista** con la enfermedad de hernias, haciendo uso de la función mostrar de la clase **Lista**.

- **K: Buscar en las listas el paciente con apendicitis de menor prioridad y el de hernia con mayor prioridad.**

Busca dentro de la lista de pacientes con apendicitis el paciente que ocupa el primer lugar dentro de la **Lista** perteneciente al quirófano de apendicitis y lo muestra por pantalla, luego de igual manera pero con la Lista de los pacientes del quirófano con hernias muestra el último que se insertó en la Lista.

- **L: Reiniciar el programa.**

Se vacían todas las listas liberando todos los nodos, esto hace que los pacientes las salas y los quirófanos estén libres de datos.

- **S: Salir.**

Provoca la salida del programa al escapar de un bucle **While** que mantiene encendida la función status de la clase controlador y la función elefirOpcion que dispara las funciones anteriormente descritas.

# Explicación de los métodos más destacados

### Controlador::opcionA()

La **opción** A del controlador genera automáticamente 12 pacientes con datos aleatorios, básicamente un bucle con 12 repeticiones comprobando el límite máximo de la **Pila** (48 pacientes)

```
for (i = 0; i<bucle_maximo ; i++)
{
    if (pila_pacientes.getLongitud()<pila_maximo)//
    {

        Paciente paciente_creacion = Paciente(); //
        if(paciente_creacion.getEnfermedad() == 1)//
            {
                while(!bucle)
                {
```

Además dependiendo de la enfermedad que padece el paciente **Apendicitis** (enfermedad == 1) o **Hernias** (enfermedad == 0) entra a un bucle de validación donde se le calcula de forma pseudoaleatoria un **ID** y una habitación únicos dentro del rango permitido, una vez comprobado que no existan datos repetidos o que colisionan, se inserta en la **Pila**.

Explicando en más profundidad, una vez generado el paciente con la función **rand()** generamos su identificador que dependiendo del valor que tenga la enfermedad tendrá un rango u otro, siendo los siguientes:

- Apendicitis -> Número aleatorio entre 49 y 1.
  **id_comprobar = rand() % 49 + 1;**
- Hernias -> Número aleatorio entre 51 y 99.
  **id_comprobar = rand() % 49 + 51;**

Para el valor de las habitacion tambien usamos la misma logica que con los **ID** lo único que cambia es el valor que puede tomar, siendo los siguientes:

- Apendicitis -> Número aleatorio **(rango 101-199 segun la practica)**
  **habitacion_comprobar = rand() % 99 + 101;**
- Hernias -> Número aleatorio **(rango 201-299 segun la practica)**
  **habitacion_comprobar = rand() % 99 + 201;**

Luego por último comprobamos que los **ID** y las habitaciones no se repitan, lo que estará generando en un bucle while, **IDs** y números de habitaciones, en caso de que se lleguen a repetir, de pasar la comprobación y no encontrar datos repetidos corta el bucle saltando al siguiente paciente.

```
while(!bucle)
{


    id_comprobar = rand() % 49 + 1;//creamos un numero random entre 49 y 1
    habitacion_comprobar = rand() % 99 + 101;//generamos la habitacion random de apendicitis (rango 101-199 segun la practica)
    if (id_comprobar != pila_pacientes.getPacienteId(id_comprobar).getId() && habitacion_comprobar != pila_pacientes.getPacienteHabitacion(habitacion_comprobar).getHabitacion())
    {
        paciente_creacion.setId(id_comprobar);//le asignamos el id al paciente creado
        paciente_creacion.setHabitacion(habitacion_comprobar); //asignamos la habitacion
        pila_pacientes.insertar(paciente_creacion); //lo insertamos
        break; //salimos del while
    }
}
```

### Paciente::randomDni()

```
Paciente::Paciente()
{
    dni = randomDni(); //declarar e
    enfermedad = randomEnfermedad();
    habitacion = 0;
    identificador = 0;
```

Es una función que nos genera de manera ficticia un **DNI** siguiendo las reglas básicas que son:

**8 cifras aleatorias y una letra al final**.

Emplea la función estándar **rand()** para obtener un número entero aleatorio de 8 cifras (entre 10.000.000 y 99.999.999)

**int numeros = rand() % 90000000 + 10000000**

Posteriormente, selecciona de forma aleatoria un carácter dentro de una cadena de strings que contiene las 26 letras del abecedario.

**string letras = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";**
**char letra = letras[rand() % 26];**

Concatenando ambos valores en un único objeto string, por ejemplo: **48392017D**

**return to_string(numeros) + letra;**

### Controlador::opcionD()

```
while(pila_pacientes.getUltimo())
{
    int longitud = pila_pacientes.getLongitud();
    int i;
    Paciente paciente;
    for(i = 0; i<longitud; i++)
    {
        paciente = pila_pacientes.extraer();

        if(paciente.getEnfermedad() == 0)
        {
            if(cola_C.getLongitud() <= cola_D.getLongitud())
                cola_C.insertar(paciente);
            else
                cola_D.insertar(paciente);
        }
        else if(paciente.getEnfermedad() == 1)
        {
            if(cola_A.getLongitud() <= cola_B.getLongitud())
                cola_A.insertar(paciente);
            else
                cola_B.insertar(paciente);
        }
    }
}
```

Simulando la llegada de los pacientes trasladando desde la **Pila** hacia las **Colas** los pacientes que se generaron antes, dependiendo además de la enfermedad que este tenga se le **asignará una de las Salas**, siendo la distribución la siguiente:

- **Salas A y B para Apendicitis.**
- **Salas C y D para Hernias.**

Insertando al paciente en la sala que contenga el menor número de pacientes, así de esta manera estas las salas equilibradas y no colapsadas.

Para esto se va comprobando las longitudes de la **Colas** correspondientes con la función **.getLongitud()**, como vemos en la imagen si la *cola_c* es menor o igual que la *cola_d* metemos el paciente en la *cola_c* pero si es mayor lo metemos en *cola_d* para mantener el equilibrio y orden en las **Colas**.

## Controlador::opcionH()

Esta función se encarga de extraer los pacientes de las salas de espera, es decir, las **Colas** y dependiendo del nivel de gravedad de su enfermedad que está ligado al **identificador/ID** que se les asignó a cada paciente los va insertando en las **Listas** siendo que el paciente de mayor gravedad quede al final de la **Lista**. Comienza evaluando desde el **ID** más grave posible **(id_apendicitis = 49)** e irá descendiendo hasta el 1.

```
int id_apendicitis = 49; //empezamos por el id mas grande posible
while((cola_A.getPrimero() != NULL || cola_B.getPrimero() != NULL) && id_apendicitis >= 1)
{
    //buscamos en la cola A el paciente con el id actual
    Paciente paciente_a_lista = cola_A.getPacienteId(id_apendicitis); //consulta el pacien
    if(paciente_a_lista.getId() != -1) //si no es el sentinela, lo hemos encontrado en la
    { ...
    else
    { ...
    id_apendicitis--; //bajamos un escalon de gravedad
}
```

Esto gracias a un bucle while, que va revisando en Las **Colas**, es decir, que estará revisando si el paciente cuyo identificador es 4 si está en la Sala A o en la Sala B. Si el paciente es hallado en la **Cola** A (es decir, el **ID** retornado no es un valor **centinela -1**), se inserta en la lista de apendicitis mediante:

```
Paciente paciente_a_lista = cola_A.getPacienteId(id_apendicitis);
if(paciente_a_lista.getId() != -1) //si no es el sentinela, lo he
{
    lista_apendicitis.insertarHead(paciente_a_lista); //lo metemo
    cola_A.borrarPorId(id_apendicitis); //lo extraemos de la cola
}
else
{
```

**lista_apendicitis.insertarHead(paciente_a_lista)**

Se borra de la **Cola** de origen para evitar duplicados, si no se encuentra en la **Cola** A, se realiza la misma operación de búsqueda y extracción en la **Cola** B.

Esta lógica se aplica tanto para todas las Colas, así que los pacientes con **Apendicitis** o **Hernias** pasan el mismo barrido, lo único que cambia es a la **Lista** a las cuales serán insertados.

Universidad de Alcalá

# Bibliografía


Números aleatorios C++

Clases en C++

Getters y Setters en C++

Shortcuts en CodeLite

GitHub Desktop

Entorno de CodeLite